

Types construits : p-uplets, tableaux, dictionnaires

Ce que je vais savoir faire

- ⊙ C1 — Je sais utiliser des p-uplets et écrire une fonction qui renvoie plusieurs valeurs.
- ⊙ C2 — Je sais lire, modifier et construire des tableaux, y compris en compréhension.
- ⊙ C3 — Je sais représenter et parcourir une grille avec un tableau de tableaux.
- ⊙ C4 — Je sais créer, modifier et parcourir un dictionnaire.
- ⊙ C5 — Je sais prévoir l'effet d'une affectation sur un objet mutable.

CARTE DU CHAPITRE

Prérequis : variables, boucles, fonctions, chaînes de caractères. Ce chapitre mène au mini-projet : classement d'un tournoi de e-sport.

Temps estimés : ♦ 12h ♦♦ 10h ♦♦♦ 9h

0.1 Les p-uplets (tuples)

DÉFINITION D1

Un **p-uplet** (ou *tuple*) est une séquence ordonnée de p éléments, éventuellement de types différents, séparés par des virgules et entourés de parenthèses. Un p-uplet est **non mutable** : une fois créé, on ne peut ni ajouter, ni retirer, ni modifier ses éléments.

| Un p-uplet est une collection ordonnée et non modifiable de valeurs. En Python, on utilise le mot anglais *tuple*.

Exemple. Le tuple `station = ("Tunis-Carthage", 36.8, 10.2)` contient trois éléments : une chaîne de caractères et deux flottants.

```
1 station = ("Tunis-Carthage", 36.8, 10.2)
2 nom = station[0]           # "Tunis-Carthage"
3 latitude = station[1]      # 36.8
```

Contre-exemple. Toute tentative de modification provoque une erreur.

ERREUR FRÉQUENTE

Modifier un tuple comme une liste (EF1). Un tuple est immuable par conception : si l'on a besoin de modifier des données, il faut utiliser une liste.

Copie fautive : `station[1] = 37.0`

Correction : créer un nouveau tuple ou utiliser une liste.

DÉFINITION D2

La fonction `len(t)` renvoie le nombre d'éléments d'un tuple `t`. L'opérateur `in` teste l'appartenance d'une valeur à un tuple.

THÉORÈME Déballage

Une fonction peut renvoyer un tuple pour communiquer *plusieurs résultats* à la fois. On parle de **déballage** (*unpacking*) lorsqu'on affecte chaque élément du tuple à une variable distincte : `x, y = milieu(a, b)`.

POUR ALLER PLUS LOIN

Un tuple de longueur 1 s'écrit `(42,)` avec une virgule obligatoire. Sans virgule, `(42)` est simplement l'entier 42 entre parenthèses. Le tuple vide s'écrit `()`.

MÉTHODE M1 — Choisir entre tuple et liste

Quand l'utiliser. Tu hésites entre un tuple et une liste pour stocker tes données.

La méthode pas à pas.

1. Les données doivent-elles rester fixes après création ? → **tuple**.
2. Faut-il ajouter, supprimer ou modifier des éléments ? → **liste**.
3. Les éléments sont-ils de types différents (nom + âge + note) ? → **tuple**.
4. S'agit-il d'une collection homogène (liste de scores) ? → **liste**.

Exemple rédigé. Coordonnées GPS d'une station : ("Tunis", 36.8, 10.2) → tuple (la position ne change pas). Relevés de température horaires : [18, 20, 19] → liste (on ajoute des mesures).

Les pièges.

- Ne pas utiliser une liste quand les données sont fixes.
- Ne pas oublier la virgule pour un tuple d'un seul élément.

Vérifier son résultat. Si tu as choisi un tuple, vérifie que tu ne tentes jamais de le modifier.

S'entraîner. Exercices 2, 4, 8.

Exercices

Exercice 1

◆ SPEC-EX-001 — 5 min

On donne le tuple `station = ("Tunis", 36.8, 10.2)`.

1. Donner le type de `station` et le nombre de ses éléments.
2. Donner la valeur de `station[0]` et de `station[2]`.
3. Peut-on écrire `station[1] = 37.0` ? Justifier.

Exercice 2

◆◆ SPEC-EX-002 — 12 min

Un tournoi de e-sport enregistre les scores de 5 manches :

`scores = [150, 200, 180, 210, 170]`

1. Écrire une fonction `au-dessus(scores, seuil)` renvoyant le tableau des scores strictement supérieurs au seuil.
2. Écrire une fonction `moyenne(scores)` renvoyant la moyenne.

Exercice 3

SPEC-EX-003 — 20 min

Écrire une fonction `historique_scores` (`equipes`, `resultats`) qui met à jour les scores d'une liste d'équipes **sans modifier** la structure d'origine. Justifier le choix de copie et analyser la complexité.

Corrigé

SPEC-EX-001

- `type(station)` renvoie `<class 'tuple'>`. Le tuple contient 3 éléments : `len(station)` vaut 3.
- `station[0]` vaut "Tunis", `station[2]` vaut 10.2.
- Non : un tuple est non mutable. L'instruction provoque `TypeError`.

QCM — Types construits

Pour chaque question, une seule réponse est correcte.

Q1. (C1) Que renvoie `type((3, 5))` ?

- `<class 'list'>` Si A : tu confonds tuple et liste $\rightarrow M1$.
- `<class 'tuple'>`
- `<class 'int'>` Si C : la virgule crée un tuple $\rightarrow D1$.

Q2. (C5) `a = [1, 2]; b = a; b.append(3); print(a)`. Qu'affiche ce programme ?

- `[1, 2]` Si A : $b = a$ crée un alias $\rightarrow D11$.
- `[1, 2, 3]`
- Une erreur

Code de référence

Fonction renvoyant un tuple

```
1 def milieu(a, b):
2     """Renvoie le milieu de deux points."""
3     mx = (a[0] + b[0]) / 2
4     my = (a[1] + b[1]) / 2
5     return (mx, my)
6
7 x, y = milieu((1, 2), (5, 8))
8 print(x, y) # 3.0 5.0
```

`>>> milieu((0, 0), (6, 8)) (3.0, 4.0) >>> type(milieu((0, 0), (6, 8))) <class 'tuple'>`

Classement d'un tournoi de e-sport

Un tournoi oppose 8 joueurs sur 5 manches. Chaque joueur est un dictionnaire avec les clés "pseudo", "equipe" et "scores".

1Représenter les données : tableau de 8 dictionnaires. 2Calculer les totaux avec une fonction `total(joueur)`. 3Trier le classement par total décroissant (copie, pas alias). 4Afficher le classement formaté.

Jalon 1 : accès aux données OK ; Jalon 2 : totaux corrects ; Jalon 3 : original non modifié ; Jalon 4 : affichage complet.